

Машинное обучение

Обучение с учителем

Методы искусственного интеллекта

Лекция 4

Жизненный цикл проекта машинного обучения

1. Понимание бизнес-цели.
Что новая система даст бизнесу?
2. Понимание цели технического проекта.
Что поступает на вход, что является целевым признаком, как оценить качество?
3. Сбор и подготовка данных.
4. Конструирование признаков.
5. Обучение модели (**мы находимся здесь**).
6. Оценка качества модели.
7. Развертывание модели.
8. Выполнение модели.
9. Мониторинг модели.
10. Сопровождение модели.

Обучение модели

Обычно на обучение модели требуется не более 10 % времени.

Основные временные затраты приходятся на этапы:

- сбора данных,
- подготовки данных,
- конструирования признаков.

Обучение модели обычно включает использование возможностей некоторой библиотеки и подбор гиперпараметров.

Обучение с учителем

Обучение с учителем (supervised learning) – один из способов машинного обучения, при котором формируется статистическая модель, обученная на данных вида «**входные признаки-целевой признак**».

Полученная модель позволяет обнаружить неизвестную зависимость между входными и целевым признаком.

Обучение сводится к минимизации средней ошибки отклонения полученного целевого признака от реального значения целевого признака для всей выборки.

Задачи обучения с учителем:

1. Регрессия (аппроксимация).
Получение значения из области значений целевого признака.
2. Классификация.
Получение метки класса (выбор из конечного множества значений).

Условия применения обучения с учителем

Условия для применения моделей обучения с учителем:

1. Имеется размеченный набор данных.
2. Произведено разбиение набора данных на выборки.
3. Примеры в наборах статистически похожи.
4. Процессы предобработки данных и конструирования признаков выполнялись только на основе данных обучающей выборки.
5. Для всех наблюдений построены векторы признаков (в основном числовые).
6. Выбрана метрика оценки качества модели.
7. Выбран ориентир.

Разбиение данных

Для обучения с учителем необходимо выполнить разбиение набора данных на выборки.

Особенности процесса разбиения данных:

1. Статистическое распределение данных в выборках должно совпадать.
2. Статистическое распределение набора данных должно совпадать с распределением в производственной среде.

Существует возможность возникновения сдвига распределения.

Определение достижимого уровня качества модели

Определение достижимого качества модели позволяет понять, когда стоит закончить этап обучения.

Когда модель может достичь «хорошего» уровня качества:

1. Если разметка данных не требует от человека применения сложных средств и методов.
2. Если признаки содержат все необходимые сведения для получения целевого признака.
3. Если вектор входных признаков содержит большое количество сигналов (применимо для изображений и текстов).
4. Если существует программная система, способная решать поставленную задачу.

Также следует выбрать **одну метрику оценки качества** модели и использовать ее для оценки.

Выбор ориентира

Перед началом обучения модели следует выбрать ориентир для сравнения уровня качества полученной модели.

Примеры ориентиров:

1. Показатель качества решения задачи человеком.
2. Показатель качества существующего алгоритма или модели.
Целевые признаки должны совпадать!
3. Алгоритм случайного предсказания.
Выбор случайного значения из области значений целевого признака или случайного класса.
4. Алгоритм правила нуля.
Выбор среднего значения из области значений целевого признака или класса с наибольшим количеством наблюдений.

Свойства алгоритма обучения

1. Объяснимость.

При повышении качества модели снижается степень понимания и объяснимости полученных результатов (черный ящик).

2. Требования к объему оперативной памяти.

3. Требования числу признаков и примеров.

4. Возможность работы с нелинейными данными (линейное разделение).

5. Скорость обучения.

6. Скорость предсказания.

Выбор алгоритма обучения

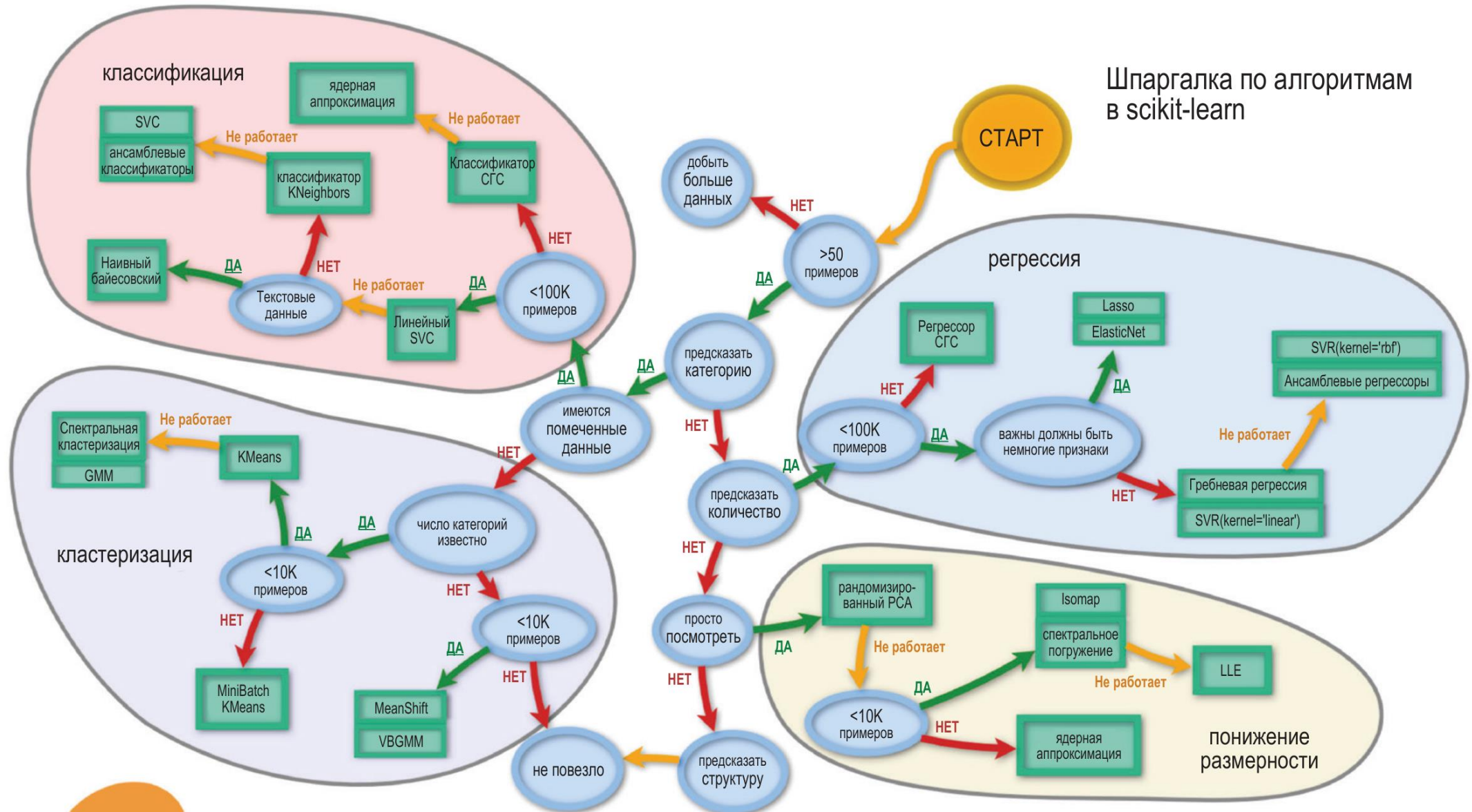
Выборочная проверка алгоритмов (algorithm spot-checking) заключается в определении списка алгоритмов-кандидатов для решения задачи.

Рекомендации:

1. Алгоритмы должны быть основаны на разных принципах.
2. Тестирование каждого алгоритма с несколькими вариантами гиперпараметров.
3. Использование фиксированного набора выборок для всех алгоритмов.
4. Если алгоритм недетерминированный, то необходимо провести несколько экспериментов, а затем усреднить результаты.
5. Выполнить документирование результатов для использования в будущих проектах.
6. Лучше провести тестирование большего количества различных алгоритмов, чем пытаться повысить качество одного, но хорошо известного алгоритма.
7. Лучше использовать библиотеки алгоритмов (scikit-learn).

Выбор алгоритма обучения

Шпаргалка по алгоритмам
в scikit-learn



Построение конвейера

Конвейеры позволяют автоматизировать следующие процессы:

1. Предобработка данных.
2. Конструирование признаков.
3. Понижение размерности признакового пространства.
4. Обучение модели.

Полученный конвейер можно сохранить в файле и использовать в производственной среде.

См. пример.

Оценивание качества моделей

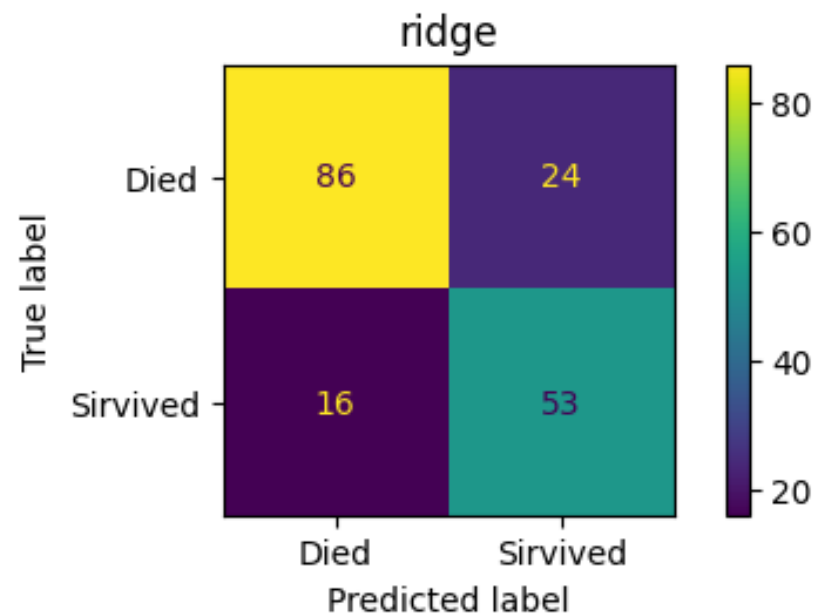
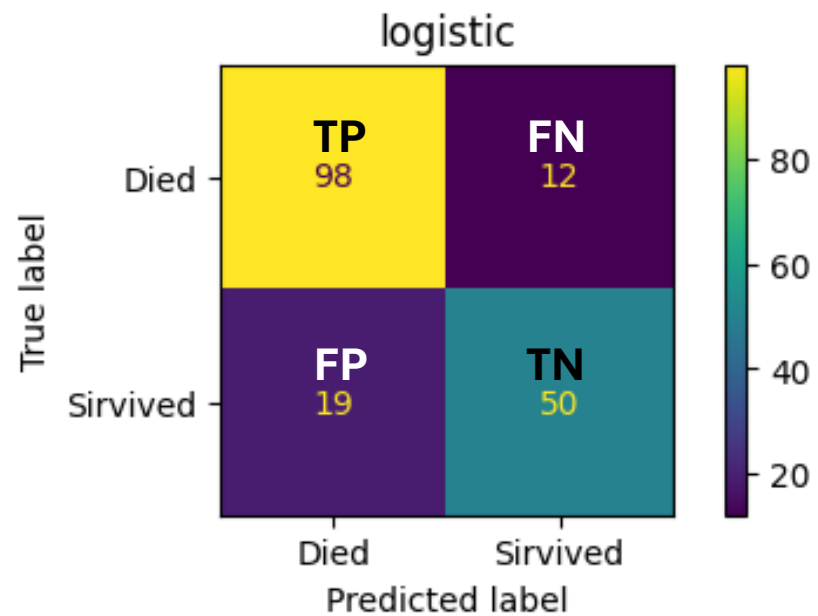
Метрики оценки качества для регрессии:

- Среднеквадратическая ошибка (mean squared error (MSE));
- Медианное абсолютное отклонение (median absolute error (MdAE));
- Частота почти правильных предсказаний (almost correct predictions error rate (ACPER)) – процентная доля предсказаний, отклоняющихся от истинного значения не более чем на процент p и др.

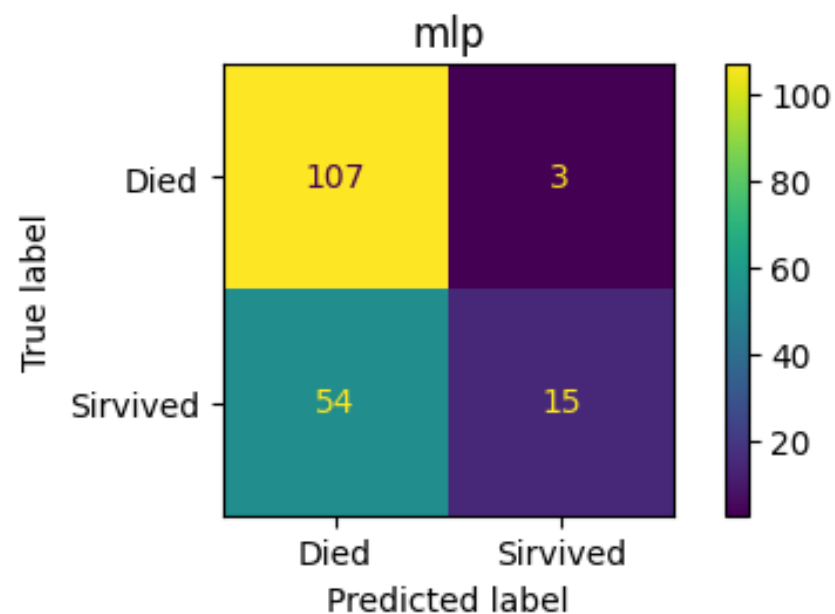
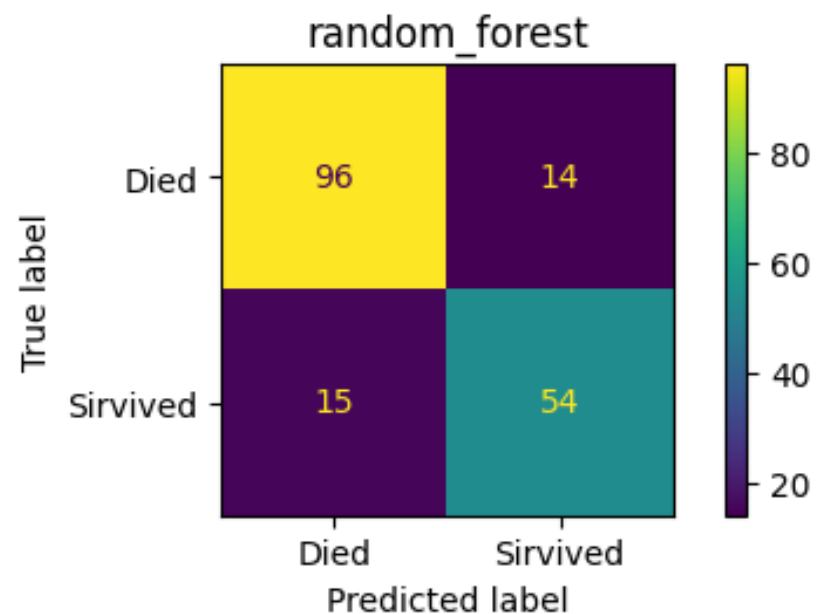
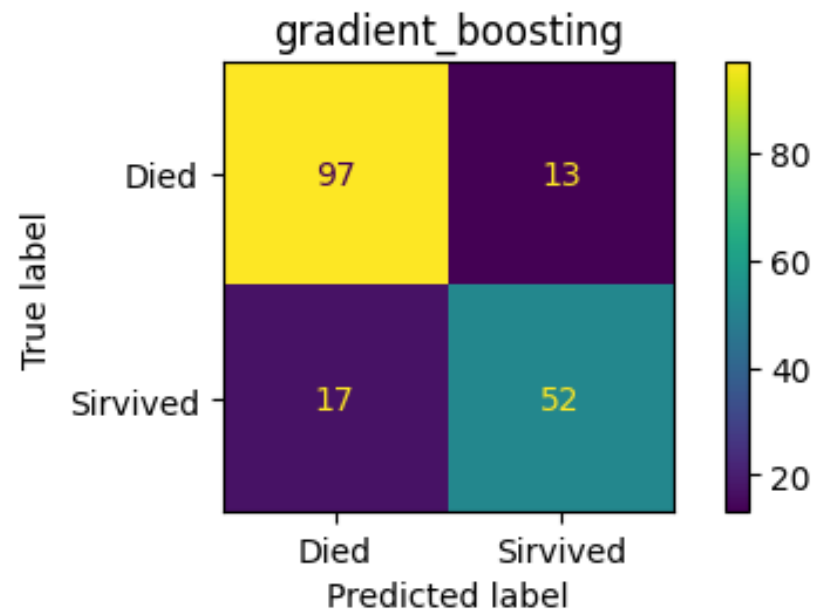
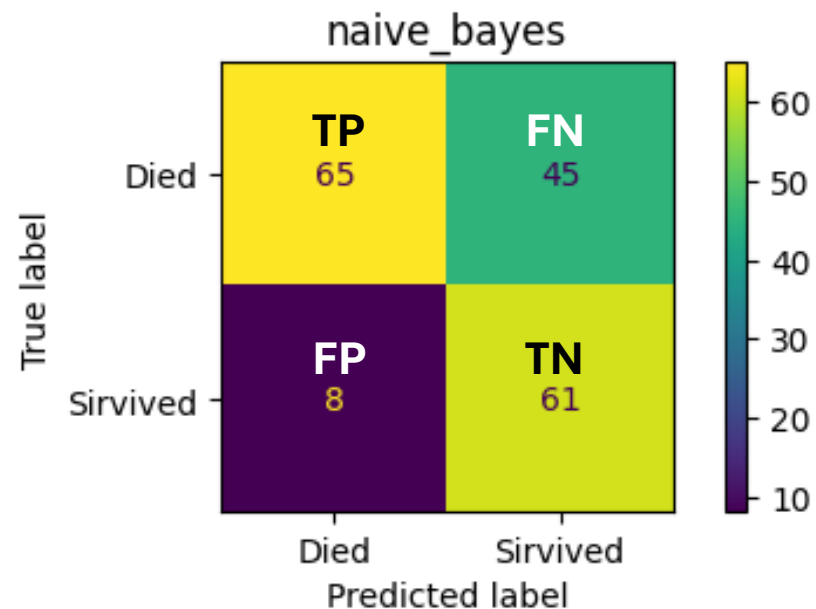
Метрики оценки качества для классификации:

- Матрица неточностей (confusion matrix);
- Точность-полнота (precision-recall);
- F1-мера (F1-score);
- Верность (аккуратность) (accuracy);
- Каппа Коэна (Cohen's kappa);
- Корреляция Мэтьюса (Matthew's correlation coefficient, MCC);
- ROC-кривая (Receiver operating characteristic curve, ROC-curve) и др.

Матрица неточностей - 1



Матрица неточностей - 2



Точность, полнота и F1-мера

Точность (precision) – отношение истинно положительных предсказаний к общему числу положительных *предсказаний*.

$$precision = \frac{TP}{TP + FP}$$

Полнота (recall) – отношение истинно положительных предсказаний к общему числу положительных *примеров*.

$$recall = \frac{TP}{TP + FN}$$

F1-мера (F1-score) – среднее гармоническое точности и полноты.

Невозможно добиться высокой точности и полноты одновременно!

$$F_1 = 2 \times \frac{precision \times recall}{precision + recall}$$

Верность (аккуратность), каппа Коэна

Верность (аккуратность, accuracy) – число правильно классифицированных примеров, поделенное на общее число классифицированных примеров.

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Верность стоит использовать, когда ошибки предсказания всех классов считаются одинаково важными.

Каппа Коэна (Cohen's kappa) – каппа Коэна определяет, насколько построенная модель работает лучше классификатора, который случайным образом предсказывает класс.

$$kappa = \frac{p^p - p^r}{1 - p^r}$$

где p^p – результат модели, p^r – фактические данные.

Значение каппы Коэна всегда меньше или равно 1:

- ≤ 0.6 – «плохая» модель;
- ≥ 0.61 and ≤ 0.8 – «хорошая» модель;
- ≥ 0.81 – «очень хорошая» модель.

МСС и ROC-кривая

Корреляция Мэтьюса (МСС) можно рассматривать как улучшенный вариант F1-меры, так как позволяет работать с несбалансированными классами.

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

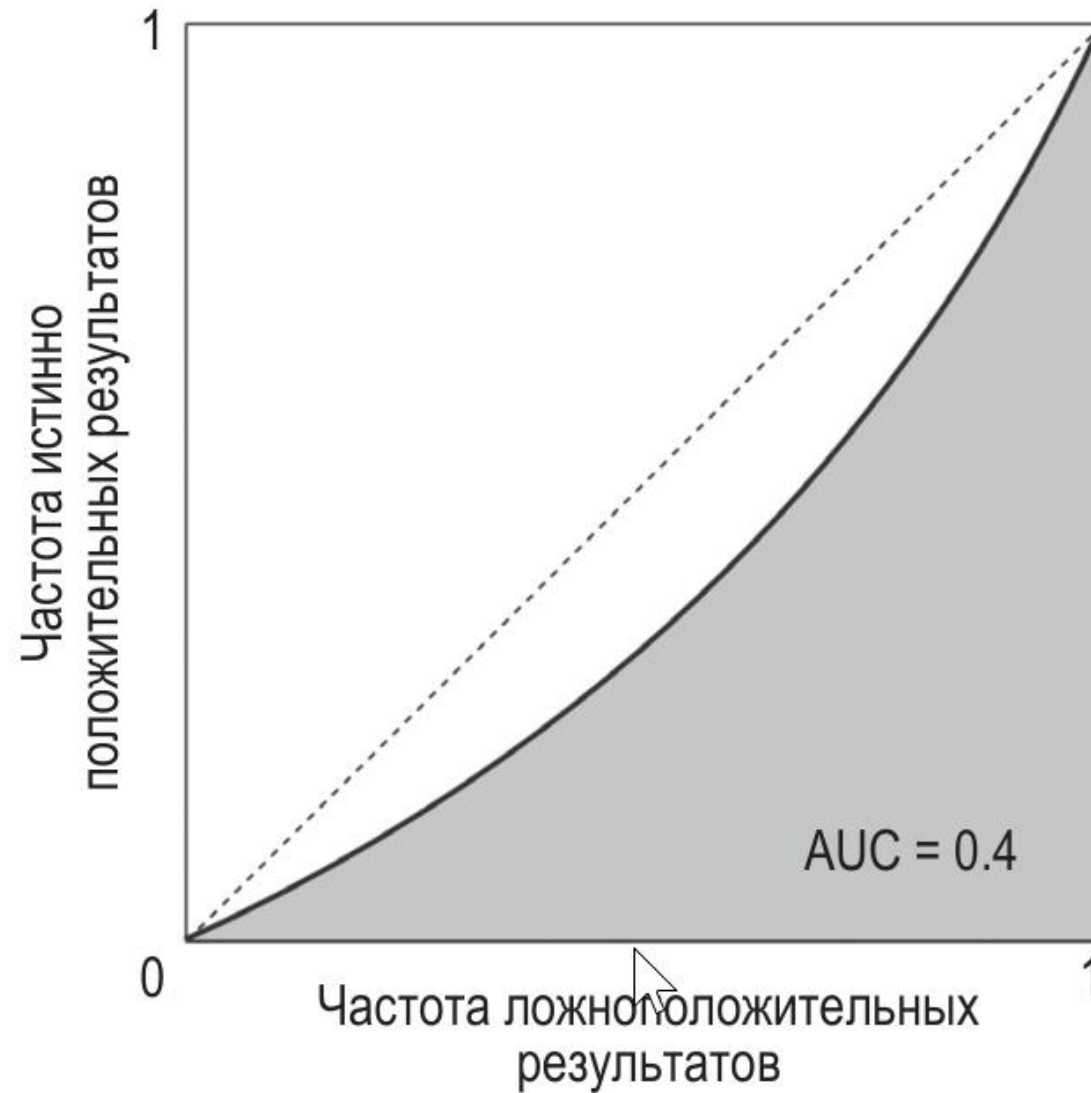
Имеет значение от -1 до 1:

- -1 – обратный результат предсказания;
- 0 – аналогично случайному предсказанию;
- 1 – превосходный результат предсказания.

ROC-кривая (ROC-curve) – работает только с методами, которые возвращают результаты предсказания в виде значений вероятности.

В качестве параметра оценки используется значение площади под ROC-кривой **ROC AUC** (area under curve).

ROC-кривая и ROC AUC



Настройка гиперпараметров

Гиперпараметры могут влиять на:

- качество модели,
- время обучения модели,
- точность и полноту;
- смещение и дисперсию.

Гиперпараметры могут быть как у алгоритма обучения, так и у конвейера.

Методы настройки гиперпараметров:

- поиск по сетке (перечень дискретных параметров);
- случайный поиск (диапазон допустимых значений);
- поиск с измельчением (сначала производится случайный поиск, а потом параметры уточняются с помощью поиска по сетке);
- Байесовские методы;
- эволюционные методы;
- перекрестная проверка (кроссвалидация, cross-validation) и др.

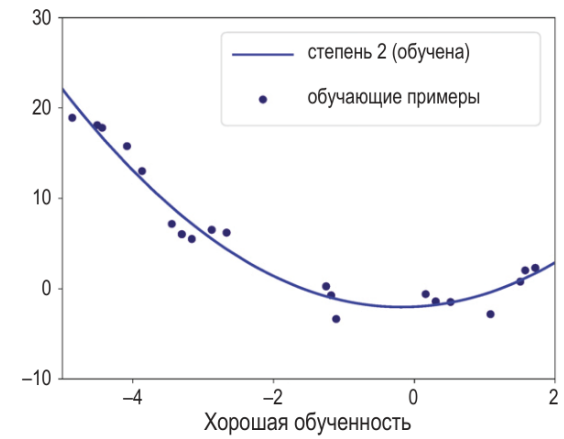
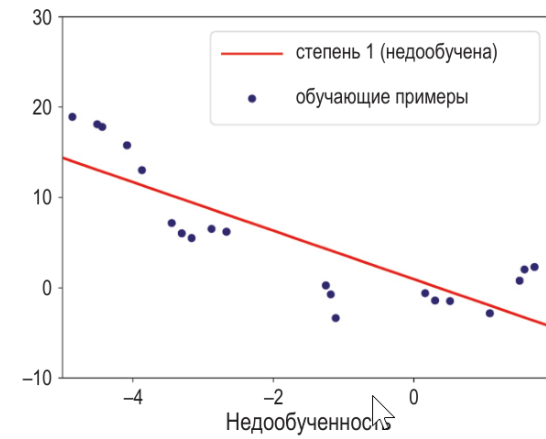
Смещение

Модель имеет низкое смещение, если модель выполняет предсказание качественно.

Если модель допускает большое количество ошибок, то такая модель имеет высокое смещение. Также говорят, что модель недообучена.

Причины недообучения модели:

- модель слишком простая для задачи и данных;
- неинформативные признаки;
- модель сильно регуляризирована.



Решение:

- использовать более сложную модель;
- построить признаки с более высокой предсказательной способностью;
- добавить больше данных в обучающую выборку;
- уменьшить регуляризацию модели.

Дисперсия

Модель имеет низкую дисперсию, если модель выполняет предсказание качественно на тестовой выборке.

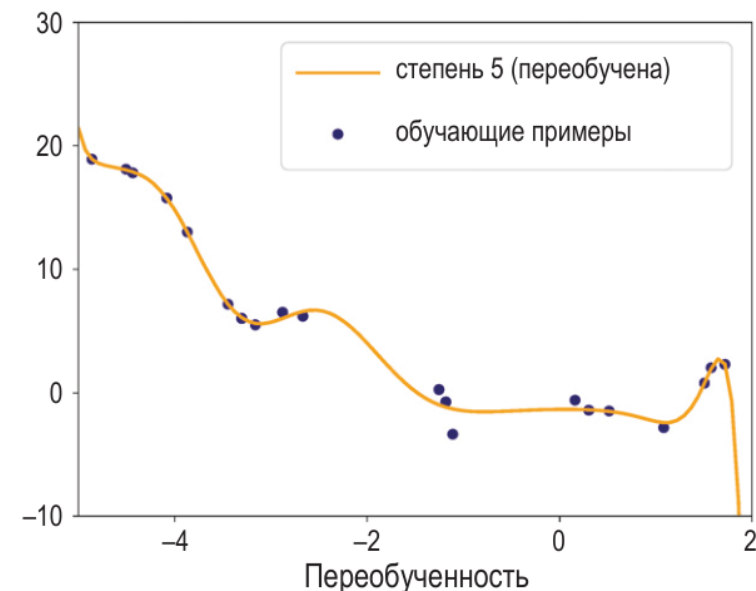
Если модель допускает большое количество ошибок на тестовой выборке, то такая модель имеет высокую дисперсию. Также говорят, что модель переобучена.

Причины переобучения модели:

- модель слишком сложная для задачи и данных;
- большое количество признаков и малое количество наблюдений;
- модель недостаточно регуляризирована.

Решение:

- использовать более простую модель;
- уменьшить размерность пространства признаков;
- добавить больше данных в обучающую выборку;
- регуляризовать модель.



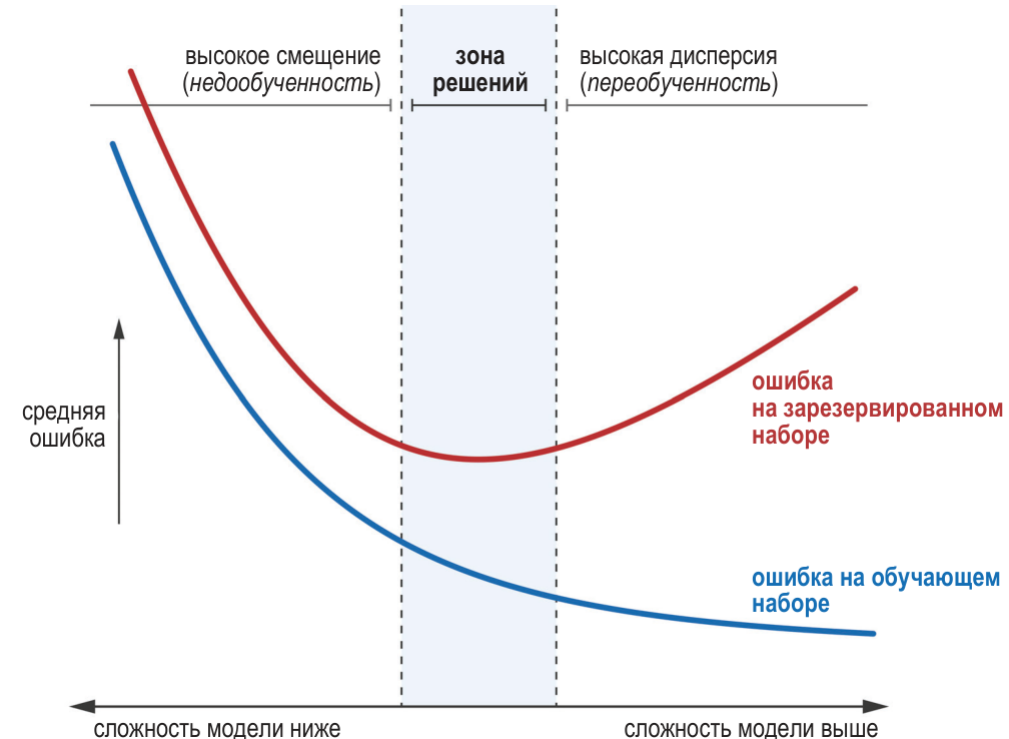
Компромисс между смещением и дисперсией

Уменьшение дисперсии приводит к увеличению смещения. Верно и обратное.

Если модель слишком сложная, то она может «запомнить» обучающие примеры и показывать высокие показатели качества на обучающей выборке, но иметь низкое качество на тестовой.

Необходимо попасть в зону решений:

- двигаться слева направо через увеличение сложности модели;
- двигаться справа налево через регуляризацию модели.



Обучение поверхностных моделей

1. Определить метрику качества.
2. Определить перечень алгоритмов обучения.
3. Выбрать метод настройки гиперпараметров.
4. Для каждого алгоритма обучения.
 1. Выбрать гиперпараметры.
 2. Обучить модель.
 3. Оценить качество модели.
 4. Продолжать пока есть варианты.
5. Выбрать модель с максимальным показателем качества.

Регуляризация

Регуляризация – методы, которые позволяют обучить более простую модель, что ведет к повышению смещения, но уменьшает дисперсию.

Существует два метода регуляризации:

1. L1-регуляризация (lasso) позволяет получить разреженную модель через выполнение отбора признаков за счет уменьшения значимости отдельных признаков.
2. L2-регуляризация (гребневая регуляризация) позволяет повысить качество модели на тестовой выборке по сравнению с L1.

Ансамблевое обучение

Ансамблевое обучение – объединение нескольких слабых моделей в сильную модель, например:

- алгоритм случайного леса;
- градиентный бустинг и др.

Принципы работы ансамблевых моделей:

1. Усреднение для регрессии.
2. Голосование для классификации.
3. Объединение моделей. Выходы слабых моделей используется для обучения сильной модели.

Методы решения проблем качества модели

1. Итеративное уточнение модели. Выполнять обучение и тестирование модели на небольшом количестве данных для выявления ошибок в данных.
2. Анализ и устранение систематических ошибок в данных: отсутствие достаточного количества примеров для отдельных сценариев.
3. Анализ и устранение ошибок: визуальный анализ результатов модели для понимания особенностей наблюдений, для которых целевой признак предсказан с ошибкой.
4. Для комплексных систем следует тестировать части отдельно на заранее подготовленных данных с высоким качеством.

Технические рекомендации

1. Следует поставлять качественную модель: имеет высокий показатель качества, не потребляет большое количество ресурсов, не завершается при анализе данных с ошибками.
2. Следует использовать популярные библиотеки с открытым исходным кодом.
3. Следует учитывать бизнес-цель при настройке гиперпараметров.
4. При изменении в данных или задаче следует обучать новую модель.
5. Следует с осторожностью использовать комплексные системы.
6. Следует писать эффективный код: компиляция, распределенные вычисления, вычисления на GPU.
7. Следует тестировать модель на старых и новых данных при их получении из внешних систем.
8. Объемная выборка данных дает лучшие результаты по сравнению со сложным алгоритмом обучения и/или формированием новых признаков.
9. Следует обеспечить воспроизводимость: `random_state`, фиксация версий библиотек, фиксация окружения.